COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	August 25, 2026	**DATE SUBMITTED:**	August 26, 2026
NAME:	Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 SEARCHING ALGORITHM

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>

const int max_size = 50;

int main() {
    int dataset[max_size];
    srand(time(0));

    // Generate random values
    for (int i = 0; i < max_size; i++) {
        dataset[i] = rand();
    }

    for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
    }

    std::cout << std::endl;
    return 0;
}
```

Observation

The code successfully creates and stores 50 random integers inside an array using the rand() function. By setting **srand(time(0))**, the program uses the current system time to generate a completely different set of numbers every time it runs. The second for loop then prints all 50 numbers across the screen separated by spaces, confirming that the dataset was generated properly and is ready for search.

Table 6-1. Data Generated and Observations.

Code

```cpp
template <typename T>
void linearSearch(T data[], int arrSize, T dataFind)
{
    // Step 1: i = 0
    int i = 0;

    // Step 2: Repeat while i < n
    while (i < arrSize)
    {
        // Step 3: If item == data[i]
        if (dataFind == data[i])
        {
            std::cout << "Searching is successful" << std::endl;
            return;
        }

        // Step 4: i = i + 1
        i++;
    }

    // Step 5: If i == n
    std::cout << "The element is not found" << std::endl;
}
```

```cpp
const int max_size = 50;

int main()
{
    int dataset[max_size];

    srand(time(0));

    for (int i = 0; i < max_size; i++)
    {
        dataset[i] = rand();
    }

    for (int i = 0; i < max_size; i++)
    {
        std::cout << dataset[i] << " ";
    }

    std::cout << std::endl;

    int dataFind;
    std::cout << "Enter number to search: ";
    std::cin >> dataFind;

    linearSearch(dataset, max_size, dataFind);
```

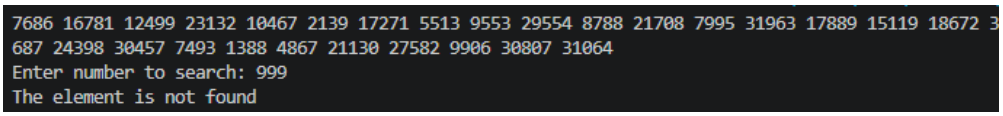
Output	
Observations	The output shows the linear search algorithm checking through the array for the user-entered number 999. Since 999 is not present in the generated random numbers, the program sequentially compares every element from start to finish without finding a match, correctly displaying "The element is not found".

Table 6-2a. Linear Search for Arrays

Code

```
template <typename T>
void linearSearchLL(Node<T> *node, T dataFind)
{
    // repeat the loop until the current node is null
    while (node != nullptr)
    {

        // if item == current node's data then display searching is succes
        if (dataFind == node->data)
        {
            std::cout << "the search is success\n";
            return;
        }

        // if not, move to the next node
        node = node->next;
    }

    //the searching reached nullptr, -> search is unsuccessful
    std::cout << "the element is not found \n";
}
```

```
// linked list: J I M L O R D
Node<char> *name1 = new_node('J');
Node<char> *name2 = new_node('I');
Node<char> *name3 = new_node('M');
Node<char> *name4 = new_node('L');
Node<char> *name5 = new_node('O');
Node<char> *name6 = new_node('R');
Node<char> *name7 = new_node('D');

// Link the nodes together
name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;

std::cout << "Linear Search for Linked List result: ";
linearSearchLL(name1, 'D');

std::cout << "Linear Search for Linked List result: ";
linearSearchLL(name1, 'X');
```

Output

```
Linear Search for Linked List result: the search is success
Linear Search for Linked List result: the element is not found
```

Observations

The code creates a singly linked list containing the letters J, I, M, L, O, R, and D, linked node by node. The linear search traverses through the pointers starting from the head node until a matching character or nullptr is found. Searching for 'D' successfully locates the character at the end of the list, whereas searching for 'X' checks every node without finding a match, correctly reporting that the element was not found.

Table 6-2b. Linear Search for Linked-list

Screenshot

```
template <typename T>
void binarySearch(T data[], int arrSize, T dataFind)
{
    int low = 0;
    int up = arrSize - 1;

    while (low <= up)
    {
        int mid = (low + up) / 2;

        if (dataFind == data[mid])
        {
            std::cout << "Search element is found!" << std::endl;
            return;
        }
        else
        {
            if (dataFind < data[mid])
            {
                up = mid - 1;
            }
            else
            {
                low = mid + 1;
            }
        }
    }

    std::cout << "Search element is not found" << std::endl;
}
```

```
int data[] = {10, 20, 30, 40, 50};

int findData;

std::cout << "Enter number to search:
std::cin >> findData;

binarySearch(data, 5, findData);
```

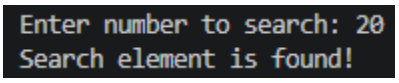
Output	
Observations	The binary search successfully found the number 20 in the array. It checked the middle values and reduced the search area until it found the correct number. This shows that binary search can find an element quickly when the array is sorted.

Table 6-3a. Binary Search for Arrays

Screenshot

```
// ========================
// BINARY SEARCH - LINKED LIST
// ========================

// Ordered linked list: 10 20 30 40 50
Node<int>* num1 = new_node(10);
Node<int>* num2 = new_node(20);
Node<int>* num3 = new_node(30);
Node<int>* num4 = new_node(40);
Node<int>* num5 = new_node(50);

// Link the nodes together
num1->next = num2;
num2->next = num3;
num3->next = num4;
num4->next = num5;
num5->next = nullptr;

// Display linked list
std::cout << "\nBinary Search for Linked List: ";

Node<int>* current = num1;

while (current != nullptr)
{
    std::cout << current->data << " ";
    current = current->next;
}

std::cout << std::endl;

int findDataLL;

std::cout << "Enter number to search: ";
std::cin >> findDataLL;

binarySearchLL(num1, findDataLL);

return 0;
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T findData) {
    //Start node is set to head of the list and last node is set to NULL
    Node<T>* last = nullptr;

    do {
        // Middle element is calculated using the two pointers approach
        Node<T>* mid = getMiddle(start, last);

        // If sub-list is empty or pointers cross
        if (mid == nullptr) {
            std::cout << "the element is not found \n";
            return;
        }

        // If middle element is same as key to be searched
        if (mid->data == findData) {
            std::cout << "the search is success\n";
            return;
        }
        // If middle element is greater than key, search right side (or if key > mid->data depending on sort order)
        // Following logic for standard ascending sorted list:
        else if (mid->data < findData) {
            start = mid->next; // Set start pointer to next of middle element
        }
        // If middle element is less than key, search left side
        else {
            last = mid; // Set last pointer to middle element
        }

    // 6. Loop breaks when start reaches or passes last node
    } while (last == nullptr || last != start);

    std::cout << "the element is not found \n";
}

#endif
```

Output

```
Binary Search for Linked List: 10 20 30 40 50
Enter number to search: 30
the search is success
```

Observations

The binary search successfully found the value 30 in the ordered linked list. It checked the middle node and narrowed down the search until it found 30. This shows that binary search can find a value efficiently when the linked list is properly ordered.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Screenshot

Array:

```cpp
#include <iostream>

int main()
{
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int key = 18;
    int comparisons = 0;

    for (int i = 0; i < 10; i++)
    {
        comparisons++;

        if (data[i] == key)
        {
            std::cout << "Found: " << data[i] << std::endl;
            break;
        }
    }

    std::cout << "Comparisons: " << comparisons << std::endl;

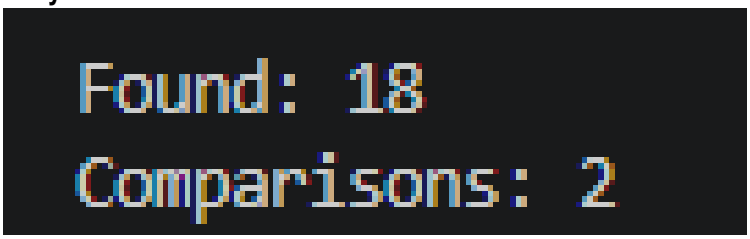
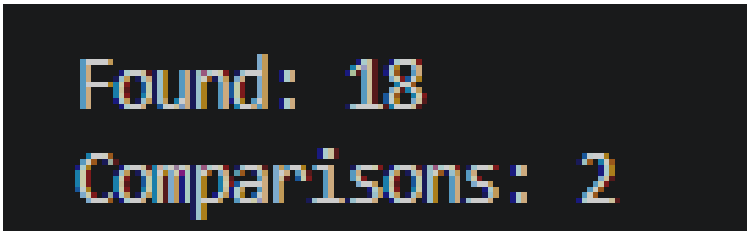
    return 0;
}
```

Linked List:

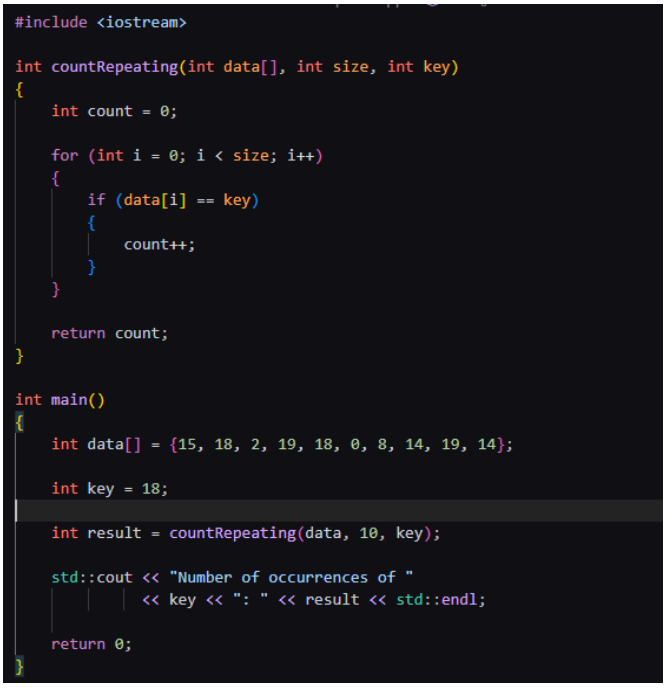
```cpp
#include <iostream>
#include "nodes.h"

int main()
{
    Node<int>* n1 = new_node(15);
    Node<int>* n2 = new_node(18);
    Node<int>* n3 = new_node(2);
    Node<int>* n4 = new_node(19);
    Node<int>* n5 = new_node(18);
    Node<int>* n6 = new_node(0);
    Node<int>* n7 = new_node(8);
    Node<int>* n8 = new_node(14);
    Node<int>* n9 = new_node(19);
    Node<int>* n10 = new_node(14);

    n1->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n5;
    n5->next = n6;
    n6->next = n7;
    n7->next = n8;
    n8->next = n9;
    n9->next = n10;
    n10->next = nullptr;
```

Output	**Array:**  **Linked List:** 
Observations	The sequential search found the number **18** after 2 comparisons. This is because 18 is the second element in the list. Both the array and linked list produced the same result.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

Screenshot	

```cpp
#include <iostream>

int countRepeating(int data[], int size, int key)
{
    int count = 0;

    for (int i = 0; i < size; i++)
    {
        if (data[i] == key)
        {
            count++;
        }
    }

    return count;
}

int main()
{
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    int key = 18;

    int result = countRepeating(data, 10, key);

    std::cout << "Number of occurrences of "
              << key << ": " << result << std::endl;

    return 0;
}
```

Output	
Observations	The search found that the number **18 appears 2 times** in the list. The algorithm checked all the elements so that it could count every occurrence. This allows it to find repeated values.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Screenshot

```cpp
#include <iostream>

int main()
{
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;
    int low = 0;
    int up = 9;
    int comparisons = 0;

    while (low <= up)
    {
        int mid = (low + up) / 2;

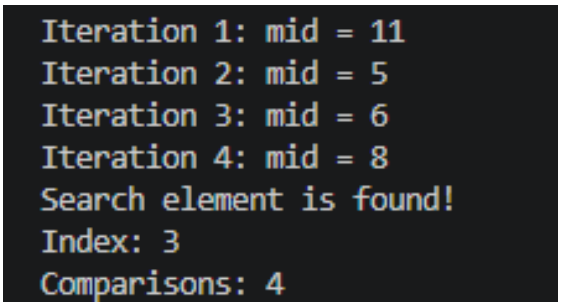
        comparisons++;

        std::cout << "Iteration " << comparisons
                  << ": mid = " << data[mid] << std::endl;

        if (data[mid] == key)
        {
            std::cout << "Search element is found!" << std::endl;
            std::cout << "Index: " << mid << std::endl;
            std::cout << "Comparisons: " << comparisons << std::endl;
            return 0;
        }
        else if (key < data[mid])
        {
            up = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    std::cout << "Search element is not found" << std::endl;

    return 0;
}
```

Output	
Observations	The binary search found the number **8** after 4 comparisons. It checked the middle value and removed half of the search area each time. This makes binary search faster when the list is sorted.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Screenshot

```cpp
#include <iostream>

int recursiveBinarySearch(int data[], int low, int up, int key)
{
    if (low > up)
    {
        return -1;
    }

    int mid = (low + up) / 2;

    std::cout << "Checking: " << data[mid] << std::endl;

    if (data[mid] == key)
    {
        return mid;
    }

    if (key < data[mid])
    {
        return recursiveBinarySearch(data, low, mid - 1, key);
    }

    return recursiveBinarySearch(data, mid + 1, up, key);
}

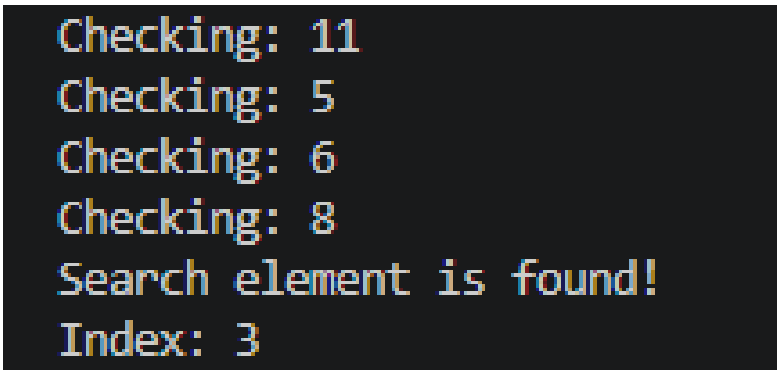
int main()
{
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;

    int result = recursiveBinarySearch(data, 0, 9, key);

    if (result != -1)
    {
        std::cout << "Search element is found!" << std::endl;
        std::cout << "Index: " << result << std::endl;
    }
    else
    {
        std::cout << "Search element is not found" << std::endl;
    }

    return 0;
}
```

Output	
Observations	The recursive binary search also found **8** at index 3. It used the same searching process as the normal binary search but called the function again for each smaller section. The result was the same as the iterative binary search.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI did not replace my critical thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.